

TRABALHO 2 · BUILDING BLOCKS · GRUPO 17 · TEMA: REPLICAÇÃO

Sistema de Arquivos Distribuído via **SMR**

Um building block genérico de Replicação de Máquina de Estados (SMR):
consistência forte e tolerância a falhas, sem depender da aplicação que roda em
cima dele.

Gustavo Rodrigues Alves D'Angelo · Tom Pereira Hunt · Pedro Henrique Gimenez

MOTIVAÇÃO

Desafios da replicação consistente

Replicar dados em vários servidores parece simples. Mas sob concorrência e falha, três problemas aparecem ao mesmo tempo.

■ Concorrência

Dois clientes escrevem ao mesmo tempo. Sem ordenação, as operações se intercalam e cada réplica termina diferente.

■ Falhas parciais

Um nó cai no meio de uma escrita. Os outros não sabem se ele aplicou ou não. O cluster pode travar esperando.

■ Assincronia

Sem relógio global: as mensagens atrasam e chegam fora de ordem, então "antes" e "depois" deixam de ser óbvios.

Quando esse controle falha, surgem **split-brain**, logs divergentes e escritas conflitantes que corrompem o estado global. O objetivo do trabalho é um mecanismo genérico de **consistência forte**, desacoplado da aplicação.

O BUILDING BLOCK

Replicação de Máquina de Estados

Mesmo estado inicial + Mesma sequência de operações = Mesmo estado final em toda réplica

01 Ordem total

Um Líder sequencia todas as propostas.
Toda réplica correta aplica o log na exata mesma ordem.

02 Determinismo

Mesma entrada produz mesma saída.
Nenhuma réplica decide nada sozinha (nem sufixo de colisão).

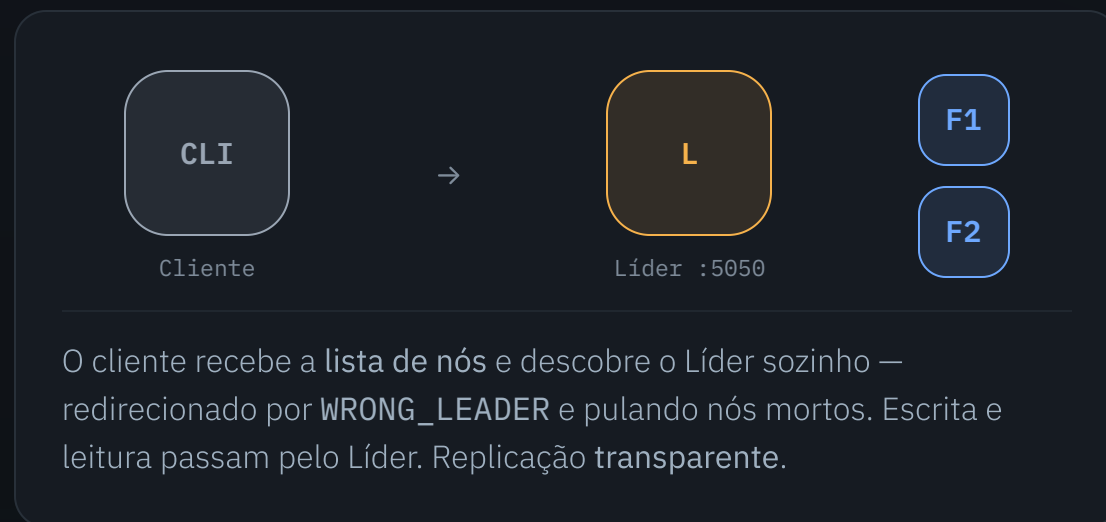
03 Desacoplamento

O motor opera sobre bytes opacos. Ele ordena pacotes. Não sabe o que é arquivo, chunk ou diretório.

O componente reutilizável é o **ReplicationNode**. Por operar apenas sobre bytes, o mesmo motor de consenso poderia replicar outra aplicação, como um banco de dados, sem qualquer alteração.

ARQUITETURA DA SOLUÇÃO

Líder-Follower em três camadas

**Camada 1 Communicator**

Sockets Berkeley TCP. Length-prefixed framing e timeouts. Resolve a fragmentação do stream.

Camada 2 ReplicationNode o building block

Consenso, ACKs, heartbeat, recuperação e gatilho de snapshot. Opera sobre bytes opacos.

Camada 3 FileSystemApp

A regra de negócio: chunking, arquivos `.part`, metadata. Implementa a interface `AppListener`.

Inversão de dependência: as camadas conversam por uma interface genérica. `on_deliver()` entrega dados, `execute_snapshot()` compacta. O consenso nunca enxerga um opcode de filesystem.

ALGORITMO DE EXECUÇÃO

Fluxo de escrita: replicação síncrona

- 1 O **cliente** envia um comando ao Líder. Único ponto de entrada.
- 2 O **Líder** pré-processa e propõe ao SMR: atribui **Epoch** e um **Sequence ID** global e único.
- 3 Transmite o frame a **todos os Followers ativos**.
- 4 Cada Follower aplica, faz **fsync** no disco e só então devolve o REPLICATION_ACK.
- 5 O Líder, que **bloqueava** nos ACKs, entrega local (on_deliver) e responde **SUCCESS**.

ACK-after-fsync

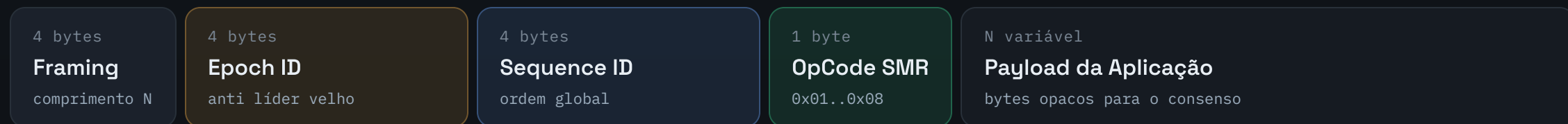
O ACK só sai depois do dado fisicamente no prato do disco. Aguenta até uma queda de energia no meio da escrita.

Não há quórum de maioria. O Líder espera o ACK de **todos os Followers ativos** e, pela **Drop Rule**, derruba quem estoura o timeout. É uma escolha por disponibilidade.

PROTOCOLO BINÁRIO

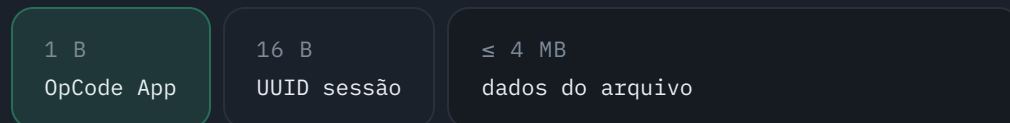
Estrutura do pacote

Comunicação direta por sockets TCP, sem HTTP ou JSON. Os pacotes são aninhados e lidos com enquadramento exato por prefixo de tamanho.



Header SMR = 9 bytes `struct "!IIB"`

0x20 Exemplo: payload de CHUNK



Por que importa

O Epoch protege contra um líder obsoleto após partição. O Sequence ID garante ordem e unicidade de cada entrada do log. Baixo overhead, leitura previsível.

Opcodes por camada

Camada de Replicação · SMR

0x01	APP_DATA	payload da aplicação
0x02	HEARTBEAT	"estou vivo"
0x03	REPLICATION_ACK	persistido com fsync
0x04	CATCHUP_REQUEST	último seq conhecido
0x05	SNAPSHOT_TRANSMIT	.tar.gz do estado
0x06	WRONG_LEADER	redireciona ao líder
0x07	I_AM_FOLLOWER	handshake de identidade
0x08	I_AM_CLIENT	handshake de identidade
0x09	WHO_IS_LEADER	sonda de descoberta
0x0A	LEADER_ANNOUNCE	líder anuncia (com epoch)
0x0B	PROMOTE	flip a líder em runtime

Camada de Aplicação · Filesystem

0x10	START	abre sessão, gera UUID
0x20	CHUNK	streama 4 MB por vez
0x30	COMMIT	rename atômico final
0x40	DELETE_FILE	remove arquivo
0x50	READ_REQUEST	pede arquivo
0x51	READ_RESPONSE	streama de volta
0x60	LIST_REQUEST	pede a árvore
0x61	LIST_RESPONSE	JSON do metadata

Encapsulamento: os opcodes de aplicação (0x10..0x61) viajam como bytes opacos dentro de um 0x01 APP_DATA. O consenso transporta sem nunca interpretar.

TOLERÂNCIA A FALHAS

Falhas e mecanismos de resposta

Falha Socket pendurado

Sem o pacote TCP FIN (queda de energia), o `recv()` do Líder travaria pra sempre.

Resposta: timeouts estritos e Heartbeat (0x02) a cada X segundos.

Falha Trava síncrona

Um Follower morre no meio do upload. O Líder bloqueado pelo ACK congelaria o cluster inteiro.

Resposta: a Drop Rule evicta o nó mudo, desbloqueia e devolve SUCCESS.

Falha Split-brain

Um líder antigo ("fantasma") volta após partição e tenta transmitir sequências obsoletas.

Resposta: o Epoch ID. Followers rejeitam tudo de um epoch inferior ao atual.

Falha Cliente no nó errado

Um cliente conecta num Follower por engano e pede uma escrita.

Resposta: WRONG_LEADER (0x06) devolve o `host:port` do líder real e o cliente reconecta.

Princípio de projeto: priorizar a **disponibilidade** em vez de exigir quórum de maioria. O cluster sobrevivente continua processando e ressincroniza os nós ausentes na recuperação.

RECUPERAÇÃO DE ESTADO

Reintegração de nós: catch-up e snapshot

Um Follower reinicia e manda `CATCHUP_REQUEST` com seu último seq. O Líder mede a lacuna e escolhe.

→ Lacuna pequena

O log ainda está na RAM do Líder. Ele faz streaming incremental só das operações perdidas. O Follower aplica em ordem e volta.

↺ Lacuna grande

O log já foi compactado. O Líder envia um snapshot `.tar.gz` (metadata.json + arquivos). O Follower restaura do zero, instantâneo.

Estado RECOVERING

Durante a restauração, broadcasts ao vivo entram numa fila `FIFO`. Só depois da extração ela é drenada. Sem corromper o estado.

Log compaction

A cada 1000 operações, snapshot e descarte do log antigo. Líder e Followers compactam deterministicamente (`seq % 1000 == 0`).

Target-bounded

O Líder responde com seu seq atual. O Follower sabe a meta e ativa sozinho, mesmo sem tráfego novo. Recuperação nunca trava.

CAMADA DE APLICAÇÃO

Sistema de arquivos distribuído

 client.py · DFS>

```
DFS> upload relatorio.pdf relatorio.pdf
  start · chunk x3 · commit ... replicado em 3 nós
DFS> list
  relatorio.pdf    1.4 MB
  foto.png         842 KB
DFS> download relatorio.pdf ./local.pdf
  ok · lido direto do Líder
DFS> delete foto.png
  ok
```

.part Atomicidade no upload

Cada upload escreve num temporário `.UUID.part`. O arquivo só vira final no `COMMIT`, com um `os.rename` atômico. Nada de arquivo meio escrito visível.

Colisão determinística

Dois "video.mp4" ao mesmo tempo? Só o Líder calcula o sufixo (1), serializado no lock do sequenciador (sem corrida), e o embute no `COMMIT`. Os Followers aplicam a string exata — nunca decidem sozinhos.

Leitura forte no caminho normal

Todo `read` e `list` é servido pelo Líder, do disco local — sem leitura stale enquanto o Líder vive. Na falha, é um trade consciente por disponibilidade.

VALIDAÇÃO · `./RUN_TESTS.SH`

Testes automatizados

O script sobe um Líder e dois Followers locais e roda o ciclo inteiro, do upload ao estado final. São dez verificações ponta a ponta, sempre do zero.

01 Replicação síncrona

Upload idêntico nos 3 sandboxes.

02 Queda de follower

SIGKILL; Drop Rule, cluster segue.

03 Catch-up incremental

Follower volta e sincroniza o que perdeu.

04 Snapshot recovery

Nó zerado restaura do `.tar.gz`.

05 Download / read

Conteúdo baixado = original, byte a byte.

06 Delete replicado

Remoção propaga aos 3 nós.

07 Colisão concorrente

Sufixo (1) determinístico, sem lost-update.

08 Diff das réplicas

sha256 dos 3 nós comparado e impresso.

09 Redirecionamento

Follower devolve `WRONG_LEADER` + host:port do líder.

10 Falha de líder

Flip em runtime (epoch+1); follower redescobre sozinho, sem reinício.

Verificação final automatizada: o sha256 dos arquivos das 3 réplicas é comparado e impresso — **IDENTICAL** em cada cenário.

CONCLUSÃO

Limitações e contribuições

Limitações (escopo acadêmico)

- ▶ **Sem eleição automática.** Sem Raft. Reconfiguração: PROMOTE faz o flip de 1 nó em runtime (epoch+1) e os followers redescobrem o líder sozinhos — nenhum nó vivo é morto.
- ▶ **Gargalo no Líder.** Toda leitura e escrita passa por ele — o preço da consistência forte no caminho normal.
- ▶ **Trade CAP na falha.** A Drop Rule prioriza disponibilidade: uma escrita recém-confirmada pode viver só no Líder até o ACK dos followers.
- ▶ **Log volátil em RAM.** Sem WAL persistente. Crash total antes do snapshot pode perder ops recentes.

Resultado

- ✓ **Aplicação distribuída real** em 3 processos independentes, sockets Berkeley.
- ✓ **Recuperação completa:** incremental por log e total por snapshot.
- ✓ **Building block modular** e desacoplado, aderente aos conceitos de SMR e tolerância a falhas.

A entrega central é o motor de replicação: genérico, testável e reutilizável. O sistema de arquivos é a aplicação construída para demonstrá-lo na prática.

INE5418 · GRUPO 17 · TEMA: REPLICAÇÃO

Obrigado. Perguntas?

Gustavo Rodrigues Alves D'Angelo · Tom Pereira Hunt · Pedro Henrique Gimenez

Sistema de Arquivos Distribuído com Replicação de Máquina de Estados.